Informatika és távközlés ágazat

PhoneScout

Készítette
Pocsai Gergő
Kovács Márk
Barkó Zoltán
Szoftverfejlesztő és -tesztelő szak

Témavezető
Kerényi Róbert Nándor
oktató

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés **4**

1.1. A projekt bemutatása és célja 4

1.2. Célközönség meghatározása 5

1.3. Alkalmazott technológiák és eszközök 5

2. Követelményanalízis **6**

2.1. Funkcionális követelmények 6

2.2. Nem funkcionális követelmények 7

2.3. Használati esetek (Use Cases) 7

2.3.1. Regisztráció 7

2.3.2. Telefonok közötti szűrés 8

2.3.3. Telefonok összehasonlítása 8

2.3.4. Telefon megtekintése 9

2.3.5. Telefon szervizeltetése 9

2.3.6. Telefon vásárlása 10

2.3.7. Profil oldal kezelése 10

2.3.8. Admin felület 11

2.3.9. Partnercég felülete 11

3. Rendszertervezés **12**

3.1. Adatbázis terv 12

3.1.1. EK-diagram és kapcsolatok 12

3.2. Rendszerarchitektúra és komponensek 13

3.2.1. ORM (Object-Relational Mapping) 13

3.2.2. Entity Framework 13

3.3. REST API végpontok tervezése 14

3.4. UI/UX tervek (Wireframes) 15

4. Megvalósítás **16**

4.1. Adatbázis réteg implementációja 16

4.1.1. Adatbázis létrehozása és konfiguráció 16

4.1.2. Táblák és kapcsolatok implementációja 16

4.1.3. Entity Framework Core használata 17
4.2. REST Backend fejlesztése 17
4.2.1. Adatkezelés és Üzleti logika 17
4.2.2. Biztonság és Autentikáció 18
4.3. React Frontend fejlesztése 18
4.3.1. Komponens-architektúra 18
4.3.2. Reszponzív megoldások és Mobil nézet 19
4.4. WPF Desktop alkalmazás 19
4.4.1. MVVM architektúra és Adatkötés 19
4.4.2. Törzsadat-karbantartó modulok 19
4.5. Összegzés 20

5. Minőségbiztosítás és tesztelés **21**
5.1. Backend egységtesztelés 21
5.2. REST API tesztelés 22
5.3. WPF kliens alkalmazás tesztelése 23
5.4. React webes kliens tesztelése 24
5.5. Teszt esetek 24
5.6. Tesztkörnyezet 25
5.7. Tesztelési keretrendszerek 25

6. Rendszerkövetelmények és telepítés **27**
6.1. Webes alkalmazás 27
6.1.1. WPF Desktop alkalmazás 27
6.1.2. Megjegyzés a lokális futtatásról 27
6.1.3. A rendszer elindításának folyamata 28
6.1.4. Backend indítása 28
6.1.5. Webes frontend indítása 28
6.1.6. Adminisztrációs felület (WPF) használata 28
6.1.7. Adatbázis előkészítése lokális futtatáshoz 29

7. Összegzés **30**
7.1. Projekt értékelése 30
7.2. Továbbfejlesztési lehetőségek 30

Mellékletek **32**

1. fejezet

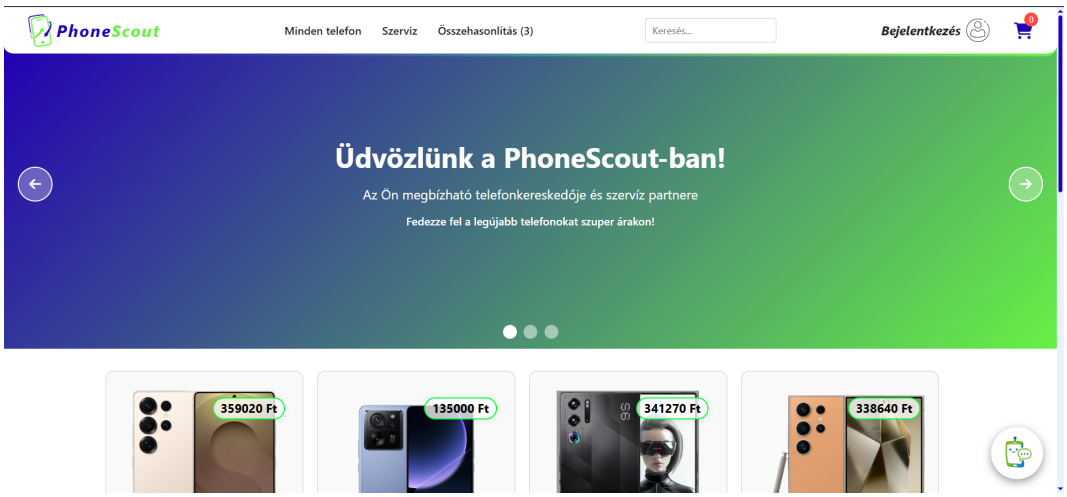
Bevezetés

1.1. A projekt bemutatása és célja

A **PhoneScout** projekt egy telefonbolt és telefonszerviz weboldala. Célja, hogy a felhasználók a lehető legegyszerűbben megtalálják az igényeikhez leginkább illeszkedő mobiltelefont. Egy-egy készülékről részletes adatok érhetők el, így a felhasználók számos paraméter alapján szűrhetnek, összehasonlíthatnak és kereshetnek.

Az oldalon elhelyezett, interaktív telefonábra segítségével a felhasználók egyértelműen megállapíthatják, hogy az adott információ a készülék mely részéhez tartozik. Ennek köszönhetően az oldal a szakkifejezések ismerete nélkül is könnyen használható (lásd: 7.1).

A rendszer a telefongyártók számára is új lehetőségeket kínál, mivel ők maguk is feltölthetik az éppen megjelenő készülékeiket. Ez biztosítja, hogy a látogatók hiteles, hivatalos forrásból származó információkhoz jussanak. A gyártók továbbá különböző eseményeiket is közzétehetik az oldalon, így a felhasználók egy újabb csatornán keresztül értesülhetnek a legfrissebb hírekről és bejelentésekről.



1.1. ábra. A főoldal.

1.2. Célközönség meghatározása

A projekt célközönsége minden olyan felhasználó, aki mobiltelefont szeretne vásárolni, készülékét javíttatni, vagy csak információra van szüksége az egyik eszközről. A weboldal felhasználóbarát megoldások alkalmazásával törekszik arra, hogy ezeket a folyamatokat a lehető legegyszerűbb és legátláthatóbb módon tegye elérhetővé.

1.3. Alkalmazott technológiák és eszközök

- **Adatbázis:** MySQL (relációs adatbázis)
- **Backend:** RESTful API (ASP.NET Core Web API)
- **Webes kliens:** React (Reszponzív kialakítás, mobil és webes nézetben is.)
- **Asztali kliens:** WPF (C#, admin felület)

2. fejezet

Követelményanalízis

Ez a fejezet a fejlesztendő webalkalmazással szemben támasztott funkcionális és nem funkcionális követelményeket, valamint a rendszer használati eseteit mutatja be.

A projekt célja egy telefonbolt és telefonszerviz teljes körű működtetésére alkalmas webalkalmazás megvalósítása, amely képes a felhasználók és azok tevékenységeinek kezelésére.

2.1. Funkcionális követelmények

- A rendszer biztosítja a felhasználók regisztrációját és bejelentkezését.
- A rendszer lehetőséget biztosít a számlázási cím megadására a regisztráció során, valamint utólag a számlázási és a szállítási címek kezelésére.
- A rendszer lehetővé teszi a telefonok közötti keresést és szűrést különböző paraméterek alapján.
- A rendszer biztosítja több telefon összehasonlítását paraméterek alapján.
- A rendszer részletes adatlapot jelenít meg a kiválasztott telefonról képekkel és műszaki adatokkal.
- A rendszer lehetőséget biztosít telefon szervizelésének igénylésére és telefon vásárlására.
- A rendszer lehetőséget biztosít a felhasználók számára saját profiljuk kezelésére.
- Az admin felület lehetőséget biztosít a telefonok, felhasználók és rendelések kezelésére.

2.2. Nem funkcionális követelmények

- A rendszer válaszideje egyik funkció esetén sem haladhatja meg a 2 másodpercet normál terhelés mellett.
- A rendszer biztosítja a felhasználói adatok védelmét. A jelszavak titkosítva kerülnek tárolásra, és az adatátvitel biztonságos csatornán történik.
- A rendszer használhatóságát úgy kell kialakítani, hogy a funkciók szakértelem nélkül is érthetők legyenek.
- A rendszer megbízható működésű, a felhasználói műveletek során nem eredményezhet adatvesztést.
- A rendszer kompatibilis a modern webböngészőkkel, valamint mobil eszközökön is használható.

2.3. Használati esetek (Use Cases)

2.3.1. Regisztráció

Szereplő: Felhasználó

Cél: A felhasználó regisztrálja magát.

Fő folyamat:

1. A felhasználó megnyitja a regisztrációs felületet.
2. Megadja a regisztrációhoz szükséges adatokat.
3. A rendszer ellenőrzi a megadott adatok helyességét és a jelszó erősségét.
4. Ellenőrzés után a felhasználó egy e-mailt kap, amelyben aktiválja a profilját.
5. A rendszer átirányítja a felhasználót a bejelentkezési felületre.
6. A felhasználó megadja az e-mail címét és jelszavát.
7. A rendszer belépteti a felhasználót az alkalmazásba.

Alternatív folyamatok:

- Ha a felhasználó a regisztráció során számlázási címet ad meg, a rendszer elmenti azt.
- Ha a megadott jelszó nem megfelelő vagy nem elég erős, a rendszer hibaüzenetet jelenít meg.

- Ha a megadott e-mail cím már használatban van, a rendszer hibaüzenetet jelenít meg.
- Ha a felhasználó nem aktiválja a fiókját, a rendszer a bejelentkezés során hibaüzenetet jelenít meg.

Utófeltétel: A felhasználó jogosult vásárlásra és szervizeltetésre.

2.3.2. Telefonok közötti szűrés

Szereplő: Felhasználó

Cél: Telefonok keresése szűrési feltételek alapján.

Fő folyamat:

1. A felhasználó megnyitja a „Minden telefon” aloldalt.
2. A rendszer megjeleníti az elérhető telefonokat és a szűrési feltételeket.
3. A felhasználó beállítja a kívánt paramétereket és elmenti a szűrést.
4. A rendszer megjeleníti a feltételeknek megfelelő telefonokat.

Alternatív folyamatok:

- A felhasználó törölheti az összes kiválasztott szűrési feltételt.
- A felhasználó kosárba vagy az összehasonlítási listába helyezheti a kiválasztott telefonokat.

Utófeltétel: A felhasználó szűrést hajtott végre a telefonok között.

2.3.3. Telefonok összehasonlítása

Szereplő: Felhasználó

Cél: Telefonok összehasonlítása kiválasztott paraméterek alapján.

Fő folyamat:

1. A felhasználó megnyitja az „Összehasonlítás” aloldalt.
2. A rendszer megjeleníti az összehasonlítási lehetőségeket.
3. A felhasználó kiválasztja az összehasonlítani kívánt telefonokat.
4. A felhasználó kiválasztja a releváns paramétereket.
5. A rendszer vizuálisan megjeleníti az összehasonlítás eredményét.

Alternatív folyamatok:

- A felhasználó törölheti az összehasonlítási feltételeket.
- A felhasználó egyesével vagy egyszerre törölheti az összehasonlításban szereplő telefonokat.

Utófeltétel: A felhasználó összehasonlította a telefonokat.

2.3.4. Telefon megtekintése

Szereplő: Felhasználó

Cél: Egy kiválasztott telefon adatainak megtekintése.

Fő folyamat:

1. A felhasználó megnyitja a telefon adatlapját.
2. A rendszer megjeleníti a telefon képeit és műszaki adatait.
3. A felhasználó tájékozódik a telefon tulajdonságairól.
4. Az alkatrészre mutatva a rendszer kiemeli a kapcsolódó adatokat.
5. Az alkatrészre kattintva az oldal a megfelelő részhez ugrik.

Alternatív folyamatok:

- A felhasználó kosárba vagy összehasonlítási listába helyezheti a telefont.

Utófeltétel: A felhasználó megtekintette a kiválasztott telefont.

2.3.5. Telefon szervizeltetése

Szereplő: Felhasználó

Cél: Telefon szervizeltetésének kezdeményezése.

Előfeltétel: A felhasználó be van jelentkezve.

Fő folyamat:

1. A felhasználó megnyitja a „Szerviz” aloldalt.
2. A rendszer megjeleníti a szervizűrlapot.
3. A felhasználó kitölti az űrlapot.
4. A rendszer ellenőrzi az adatokat és visszaigazolást küld.

Alternatív folyamatok:

- A felhasználó extra átvizsgálást kérhet.
- Hibás adatmegadás esetén a rendszer hibaüzenetet jelenít meg.

Utófeltétel: A szervizeltetés megjelenik a felhasználó profiljában.

2.3.6. Telefon vásárlása

Szereplő: Felhasználó

Cél: Telefon megrendelése.

Előfeltétel: A felhasználó be van jelentkezve, a kosár nem üres.

Fő folyamat:

1. A felhasználó megnyitja a „Kosár” aloldalt.
2. A rendszer megjeleníti a rendelési űrlapot.
3. A felhasználó kitölti az űrlapot.
4. A rendszer ellenőrzi az adatokat és rögzíti a rendelést.

Alternatív folyamatok:

– Üres kosár esetén a rendszer hibaüzenetet jelenít meg.

Utófeltétel: A vásárlás megjelenik a felhasználó profiljában.

2.3.7. Profil oldal kezelése

Szereplő: Felhasználó

Cél: A felhasználó saját adatainak kezelése.

Előfeltétel: A felhasználó be van jelentkezve.

Fő folyamat:

1. A felhasználó megnyitja a „Profil” aloldalt.
2. A rendszer betölti a felhasználó adatait.
3. A felhasználó módosíthatja személyes adatait és címeit.
4. A felhasználó megtekintheti rendeléseit és szervizeléseit.
5. A felhasználó kommunikálhat a szervizzel az éppen szervizelés alatt álló telefonról.
6. A felhasználó elfogashatjta, vagy elutasíthatja a szervizelésről kapott árajánlatot.

Alternatív folyamatok:

– Hibás adatmegadás esetén a rendszer hibaüzenetet jelenít meg.

Utófeltétel: A felhasználó kezelte saját adatait.

2.3.8. Admin felület

Szereplő: Admin

Cél: Az adminisztratív adatok kezelése.

Előfeltétel: Admin jogosultság és bejelentkezés.

Fő folyamat:

1. Az admin kezelheti a telefonokat.
2. Az admin kezelheti a felhasználókat és partnercégeket.
3. Az admin kezelheti a raktárkészletet.
4. Az admin kezelheti a rendeléseket és szervizeltetéseket.

Alternatív folyamatok:

– Hibás adatmegadás esetén a rendszer hibaüzenetet jelenít meg.

Utófeltétel: Az admin elvégezte az adatkezelési műveleteket.

2.3.9. Partnercég felülete

Szereplő: Cég

Cél: A partnercég saját adatainak kezelése.

Előfeltétel: Cég jogosultság és bejelentkezés.

Fő folyamat:

1. A cég módosíthatja saját adatait.
2. A cég kezelheti saját telefonjait.
3. A cég új telefonokat és eseményeket hozhat létre.

Alternatív folyamatok:

– Hibás adatmegadás esetén a rendszer hibaüzenetet jelenít meg.

Utófeltétel: A cég kezelte saját adatait.

3. fejezet

Rendszertervezés

A rendszertervezés célja az alkalmazás logikai és technikai felépítésének bemutatása, valamint a tervezési döntések indoklása. A fejezet ismerteti az adatbázis struktúráját, a rendszer architektúráját, az alkalmazott adatkezelési megközelítéseket, továbbá a felhasználói felület tervezésének alapelveit. A bemutatott tervek képezik a megvalósítás alapját, amely a következő fejezetben kerül részletezésre.

3.1. Adatbázis terv

Az alkalmazás működéséhez szükséges adatok tárolására egy relációs adatbázis került megtervezésre. Az adatbázis célja a rendszer által kezelt információk strukturált, konzisztens és hatékony kezelése. A tervezés során kiemelt szempont volt az adatintegritás biztosítása, valamint az adatok közötti logikai kapcsolatok egyértelmű meghatározása.

3.1.1. EK-diagram és kapcsolatok

Az adatbázis logikai modelljét egy entitás-kapcsolat (EK) diagram szemlélteti. Az ER-diagram bemutatja a rendszer főbb entitásait, azok attribútumait, valamint az entitások közötti kapcsolatokat. Az egyes entitások elsődleges kulccsal rendelkeznek, amelyek biztosítják az adatok egyedi azonosíthatóságát.

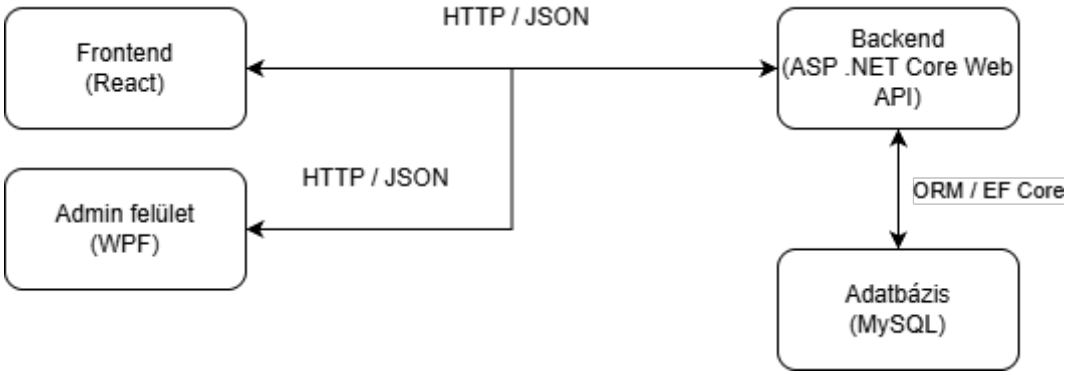
Az entitások közötti kapcsolatok idegen kulcsok segítségével kerülnek megvalósításra. A diagram tartalmazza a kapcsolatok típusát és kardinalitását, mint például az egy-egy, egy-több vagy több-több kapcsolatokat. A kialakított adatmodell megfelel a normalizációs elveknek, elősegítve az adatredundancia csökkentését.

Az adatbázisról készített EK-diagram itt található. (lásd: 7.2).

3.2. Rendszerarchitektúra és komponensek

A rendszer kliens–szerver architektúrára épül, amely elkülöníti a felhasználói felületet és a szerveroldali feldolgozást. Az architektúra fő komponensei a kliensalkalmazások, a szerveroldali API, valamint az adatbázis. A komponensek közötti kommunikáció jól definiált interfészeken keresztül történik.

A tervezett felépítés lehetővé teszi több különböző kliensalkalmazás egyidejű kiszolgálását, valamint támogatja a rendszer bővíthetőségét és skálázhatóságát.



3.1. ábra. A projekt rendszerarchitektúrája.

3.2.1. ORM (Object-Relational Mapping)

Az adatbázis és az alkalmazás közötti adatkezelés megvalósítására Object-Relational Mapping, ORM, megközelítés került kiválasztásra. Az ORM célja, hogy a relációs adatbázis struktúráját objektumorientált modellként lehessen kezelni az alkalmazásban.

Az ORM használata csökkenti az adatkezeléssel kapcsolatos kód mennyiségét, valamint hozzájárul a rendszer átláthatóságához és karbantarthatóságához. A megközelítés lehetővé teszi az adatok magasabb szintű absztrakción történő kezelését.

3.2.2. Entity Framework

A tervezés során az ORM megvalósítására az Entity Framework Core került kiválasztásra. A döntést a technológia platformfüggetlensége, széles körű funkcionalitása, valamint az ASP .NET Core környezethez való szoros integrációja indokolta.

Az Entity Framework Core támogatja az objektumorientált adatkezelést, valamint lehetőséget biztosít az adatmodell és az adatbázis szerkezetének összehangolására. A technológia alkalmazása elősegíti az egységes adatkezelési stratégia kialakítását a rendszerben.

3.3. REST API végpontok tervezése

A rendszer komponensei közötti kommunikáció REST architektúra szerint tervezett API végpontokon keresztül valósul meg. A végpontok HTTP protokollt használnak, és szabványos HTTP metódusokra épülnek az adatok lekérdezése és módosítása során.

A tervezés során meghatározásra kerültek az API végpontok logikai csoportjai, az URL struktúrák, valamint a kliens és a szerver közötti adatcsere alapelvei. A REST alapú megközelítés biztosítja a rendszer komponenseinek laza csatolását.

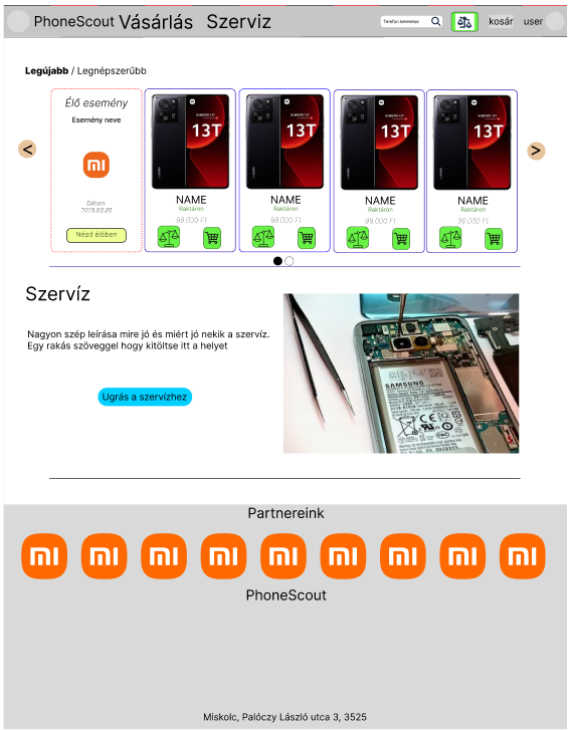
A REST API végpontok dokumentálása Swagger UI segítségével történt, amelynek egy részlete az alábbi ábrán látható.

3.2. ábra. A REST API végpontok egy részlete.

3.4. UI/UX tervek (Wireframes)

A felhasználói felület tervezése során a használhatóság és az áttekinthetőség került előtérbe. A wireframe-ek a rendszer főbb képernyőinek vázlatos elrendezését mutatják be, valamint szemléltetik a felhasználói navigáció alapvető elemeit.

A tervek célja a felhasználói folyamatok előzetes meghatározása, valamint annak biztosítása, hogy a felhasználói felület megfeleljen a funkcionális követelményeknek. A wireframe-ek nem tartalmaznak végleges grafikai elemeket, kizárólag a struktúrát és az elrendezést szemléltetik.



3.3. ábra. A főoldal terve

3.4. ábra. A telefon oldal terve

4. fejezet

Megvalósítás

A fejezet célja a korábban bemutatott rendszerterv gyakorlati megvalósításának részletes ismertetése. A megvalósítás a tervezési fázisban meghatározott architektúrák és technológiai döntések alpaján lett elkészítve, különös figyelmet fordítva a skálázhatóságra, a biztonságra és a karbantarthatóságra.

A rendszer többrétegű kliens–szerver architektúrában valósult meg, amely külön kezeli az adatbázis réteget, a backend üzleti logikát, a webes felhasználói felületet, valamint az adminisztrációs asztali alkalmazást.

4.1. Adatbázis réteg implementációja

Az alkalmazás adatkezelése relációs adatbázisban valósult meg MySQL rendszer használatával. A fizikai adatbázis létrehozása során a cél a tervezett logikai modell pontos leképezése volt.

4.1.1. Adatbázis létrehozása és konfiguráció

Az adatbázis UTF-8 karakterkódolással került létrehozásra, amely biztosítja a magyar és nemzetközi karakterek megfelelő kezelését. A táblák InnoDB tárolómotorral készültek, amely támogatja az idegen kulcsokat.

A fejlesztés során a backend adatmodellje a meglévő adatbázis alapján scaffolddal került generálásra, amely lehetővé tette a meglévő adatok könnyű használatát a fejlesztés során.

4.1.2. Táblák és kapcsolatok implementációja

Az adatbázis főbb entitásai:

- Telefon

- Felhasználó
- Rendelés
- Szervizelés

Az entitások közötti kapcsolatok idegen kulcsokkal kerültek megvalósításra. A több-több kapcsolatokat kapcsolótáblák kezelik, például a kamerák kapcsolótáblája. Az adatintegritás biztosítása érdekében minden idegen kulcs megfelelő törlési és frissítési szabályokkal rendelkezik.

4.1.3. Entity Framework Core használata

Az adatbázis és az alkalmazás közötti kapcsolat Database First megközelítésben került megvalósításra.

A fizikai adatbázis szerkezetének kialakítása manuálisan, MySQL környezetben történt, majd az Entity Framework Core eszközeivel entitásosztályok és a DbContext osztály lett legenerálva.

Az adatmodell változásai az adatbázis szerkezetének módosításával történnek, amelyet követően az entitásosztályok újragenerálásával biztosítható az alkalmazás és az adatbázis közötti konzisztencia.

4.2. REST Backend fejlesztése

A backend az ASP.NET Core Web API technológiával került megvalósításra. A rendszer REST architektúrát követ, amely biztosítja a kliens és a szerver közötti laza csatolást.

4.2.1. Adatkezelés és Üzleti logika

A backend réteg három fő logikai egységre bontható:

- Controller réteg
- Service réteg
- Adat-hozzáférési réteg

A Controller osztályok felelősek a HTTP kérések fogadásáért és a megfelelő szolgáltatások meghívásáért. A Service réteg tartalmazza az üzleti logikát, például a validációkat.

Az adat-hozzáférés az Entity Framework segítségével történik, amely LINQ lekérdezéseken keresztül kommunikál az adatbázissal.

A telefonok szűrése dinamikus lekérdezésekkel valósult meg, amely lehetővé teszi több paraméter egyidejű alkalmazását. A rendszer támogatja a lapozást (pagination) és rendezést is.

DTO (Data Transfer Object) objektumok kerültek bevezetésre az adatbiztonság és a túlpublikálás elkerülése érdekében.

4.2.2. Biztonság és Autentikáció

A rendszer biztonsági mechanizmusa token-alapú hitelesítésre épül. A jelszókezelés során kiemelt szempont volt, hogy a felhasználói jelszó titkosítatlan formában ne kerüljön továbbításra a hálózaton keresztül.

Ennek érdekében a jelszó már a kliensoldalon hash-elésre kerül egy egyedi salt értékkel kombinálva. A szerveroldalon a beérkező hash további feldolgozáson megy keresztül, ismételt hash-elés történik, majd ez az érték kerül eltárolásra az adatbázisban.

A bejelentkezés során a rendszer a kliensoldalon generált hash értéket hasonló módon dolgozza fel, majd az így előállított értéket hasonlítja össze az adatbázisban tárolt hash-el.

Ez a megközelítés biztosítja, hogy a jelszó eredeti formájában sem a hálózaton keresztül, sem az adatbázisban nem jelenik meg.

Sikeres bejelentkezést követően, a rendszer egy tokent generál.

4.3. React Frontend fejlesztése

A webes kliens React alapú, komponens-orientált architektúrával készült Node.js segítségével. A cél egy dinamikus, felhasználóbarát és reszponzív felület kialakítása volt.

4.3.1. Komponens-architektúra

Az alkalmazás moduláris felépítésű, ahol az egyes funkciók külön komponensekbe szervezve kerültek megvalósításra.

A főbb komponensek és oldalak:

- Navigációs sáv
- Telefon kártya
- Chatbot
- Szűrő oldal
- Összehasonlító oldal

– Telefon oldal

Az állapotkezelés React hook-ok segítségével történik.

Az API kommunikáció HTTP kliens segítségével történik, és axios technológiát használ a kommunikációhoz.

4.3.2. Reszponzív megoldások és Mobil nézet

A felhasználói felület kialakítása során kiemelt szempont volt a reszponzivitás. Az alkalmazás mobilnézete React Router alapú, amely viewport szélesség alapján (991px breakpoint) automatikusan vált a desktop és mobil módok között a window.matchMedia API használatával. A mobilnézet ugyanazokat az oldal komponenseket használja, mint az asztali verzió, de egy speciális MobileLayout komponenssel van becsomagolva, amely alsó navigációs sávot, hamburger menüt, és mobilra optimalizált keresőt biztosít. A megjelenés testreszabását a mobile-shell.css és mobile-functional-overrides.css fájlok kezelik, amelyek felülírják a Bootstrap és desktop stílusokat a kisebb képernyők számára.

4.4. WPF Desktop alkalmazás

Az adminisztrációs felület WPF technológiával készült `C#` nyelven. A cél egy stabil, adatintenzív műveletekre optimalizált kezelőfelület létrehozása volt.

4.4.1. MVVM architektúra és Adatkötés

Az alkalmazás MVVM (Model–View–ViewModel) architektúrát követ. A ViewModel osztályok felelősek az üzleti logika és a felhasználói felület közötti kapcsolatért.

Az adatfrissítés INotifyPropertyChanged interfész implementálásával történik. A lista jellegű adatok ObservableCollection használatával kerülnek kezelésre.

A felhasználói műveletek ICommand implementációval (pl. RelayCommand) kerültek megvalósításra.

4.4.2. Törzsadat-karbantartó modulok

Az admin felület több karbantartó modult tartalmaz:

– Telefonok kezelése (létrehozás, módosítás)

– Felhasználók kezelése (szerepkör módosítás, deaktiválás)

– Cégek kezelése (módosítás, deaktiválás)

- Rendelések státuszkezelése
- Szervizelések kezelése
- Raktárkészlet frissítése

A műveletek végrehajtása REST API hívásokon keresztül történik, biztosítva a központi adatkezelést.

4.5. Összegzés

A megvalósítás során a tervezett architektúra teljes egészében implementálásra került. A rendszer moduláris felépítése biztosítja a későbbi bővíthetőséget és karbantarthatóságot. A biztonsági mechanizmusok, a rétegzett architektúra és a modern frontend technológiák alkalmazása lehetővé teszik egy stabil és felhasználóbarát alkalmazás működését.

5. fejezet

Minőségbiztosítás és tesztelés

A szoftverfejlesztés során kiemelten fontos a rendszer stabil működésének, megbízhatóságának és hibamentességének biztosítása. Ennek érdekében a projekt fejlesztési folyamata során különböző tesztelési módszerek kerültek alkalmazásra.

A rendszer több komponensből áll: egy backend szolgáltatásból, egy webes kliens alkalmazásból, valamint egy asztali kliensből. A tesztelés ennek megfelelően több szinten történt:

- Backend egységtesztek
- REST API tesztelés
- WPF kliens funkcionális tesztelése
- React webes felület tesztelése

A tesztelési folyamat célja annak ellenőrzése volt, hogy az alkalmazás megfelele a korábban meghatározott funkcionális követelményeknek, valamint képes-e stabil működésre különböző felhasználói műveletek során.

5.1. Backend egységtesztelés

A backend alkalmazás üzleti logikájának ellenőrzése egységtesztek segítségével történt. Az egységtesztek célja az alkalmazás egyes komponenseinek elkülönített vizsgálata, amely lehetővé teszi a logikai hibák korai felismerését.

Az egységtesztek során különböző szolgáltatás osztályok működésének ellenőrzése történt, például:

- Szervizelés feltöltése
- Szervizelés módosítása

```
[Fact]
0 references
public void autoFillPCC_ValidPostalCode_ReturnsOk()
{
    var data = new List<Postalcode>
    {
        new Postalcode { Iranyitoszam = 1051, Telepules = "Budapest" }
    };

    var controller = GetControllerWithData(data);

    var dto = new PostalCodeDTO
    {
        postalCode = 1051,
        city = ""
    };

    var result = controller.autoFillPCC(dto);

    var okResult = Assert.IsType<OkObjectResult>(result);
    var value = Assert.IsType<Postalcode>(okResult.Value);

    Assert.Equal("Budapest", value.Telepules);
}
```

5.1. ábra. Backend egységteszt példája xUnit keretrendszerrel

– Város kiegészítése irányítószám alapján

Az egységtesztek segítségével biztosítható, hogy az egyes backend komponensek megfelelően működjenek, valamint a későbbi módosítások ne okozzanak újabb hibákat a rendszerben.

5.2. REST API tesztelés

A backend szolgáltatás REST API végpontjainak működésének tesztelése Swagger felületen keresztül történt. A Swagger lehetőséget biztosít a különböző HTTP kérések manuális kipróbálására és a válaszok vizsgálatára.

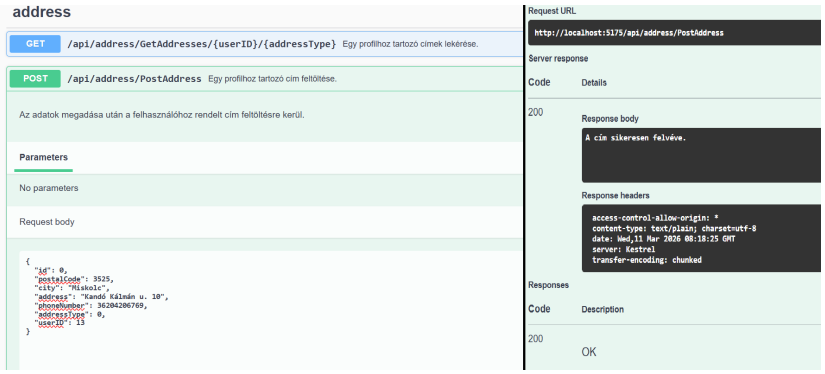
– HTTP metódusok helyes működése (GET, POST, PUT, DELETE)

– megfelelő válasz státuszkódok

– válasz objektumok szerkezete

– hibás bemenetek kezelése

– autentikáció és jogosultságkezelés



5.2. ábra. Cím feltöltés, REST API tesztelés Swagger felületen

5.3. WPF kliens alkalmazás tesztelése

A rendszer asztali kliens alkalmazása WPF technológiával készült. A tesztelés során a felhasználói felület működésének és a backend szolgáltatással történő kommunikációnak az ellenőrzése történt.

– Szervizelés adatainak validációja

```
[TestMethod]
0 references
public void PartsAreNotChanged()
{
    _originalParts = new List<string> { "Képernyő", "Akkumulátor", "Kamera" };
    Parts = new List<string> { "Akkumulátor", "Kamera", "Képernyő" };

    bool result = ArePartsChanged();

    Assert.IsFalse(result);
}

[TestMethod]
0 references
public void PartsAreChanged()
{
    _originalParts = new List<string> { "Képernyő", "Akkumulátor", "Kamera" };
    Parts = new List<string> { "Képernyő", "Akkumulátor", "Camera" };

    bool result = ArePartsChanged();

    Assert.IsTrue(result);
}

[TestMethod]
0 references
public void PartsAreChanged_WhenCountIsDifferent()
{
    _originalParts = new List<string> { "Képernyő", "Akkumulátor", "Kamera" };
    Parts = new List<string> { "Képernyő", "Akkumulátor" };

    bool result = ArePartsChanged();

    Assert.IsTrue(result);
}
```

5.3. ábra. Szerviz igénylés alkatrészlista ellenőrzése WPF kliensben

5.4. React webes kliens tesztelése

A webes kliens alkalmazás React keretrendszer használatával készült. A tesztelés célja annak ellenőrzése volt, hogy a felhasználói felület megfelelően működik különböző felhasználói műveletek során.

- Sikeres bejelentkezés végrehajt egy navigációt
- Telefonok megjelenítése a főoldalon GET kérés után
- Telefon képek betöltése
- Sikeres bejelentkezés után a token mentése local storage-ba
- Kijelentkezés után a token eltávolítása a local storage-ból

```
describe("AuthContext tesztek", () => {
  beforeEach(() => {
    localStorage.clear();
  });

  test("login menti a tokent localStorage-ba", () => {
    const { result } = renderHook(() => useAuth(), {
      wrapper: AuthProvider,
    });

    act(() => {
      result.current.login("new-token-456");
    });

    expect(localStorage.getItem("userToken")).toBe("new-token-456");
  });

  test("logout törli a tokent localStorage-ból", () => {
    localStorage.setItem("userToken", "token-1");

    const { result } = renderHook(() => useAuth(), {
      wrapper: AuthProvider,
    });

    act(() => {
      result.current.logout();
    });

    expect(localStorage.getItem("userToken")).toBeNull();
  });
});
```

5.4. ábra. React kliens – sikeres bejelentkezés és token mentése

5.5. Teszt esetek

A rendszer működésének ellenőrzésére több gyakori felhasználói művelet került tesztelésre. Az alábbi táblázat a fontosabb teszteseteket mutatja be.

Teszt neve	Bemenet	Elvárt eredmény	Eredmény
Hibás bejelentkezés	Hibás jelszó	Hibaüzenet jelenik meg	Sikeres
Sikeres bejelentkezés	Érvényes e-mail és jelszó	Navigáció a főoldalra, token mentése localStorage-ba	Sikeres
Telefon képek betöltése	Telefon kiválasztása	Telefon képek megjelennek a felületen	Sikeres
Kosár művelet	Telefon kosárba helyezése	A termék megjelenik a kosárban	Sikeres
Szerviz kezelés (WPF)	Szerviz adatinak kitöltése	Szerviz módosítás rögzítése a backendben, visszajelzés a felhasználónak	Sikeres

A tesztelési folyamat eredménye alapján a rendszer minden vizsgált funkció esetében a várt módon működött.

5.6. Tesztkörnyezet

A tesztelés az alábbi fejlesztői környezetben történt:

- Backend: ASP.NET Core Web API
- API dokumentáció és tesztelés: Swagger
- Asztali kliens: WPF
- Webes kliens: React
- Adatbázis: SQL Server

5.7. Tesztelési keretrendszerek

A rendszer különböző komponenseinek teszteléséhez az alábbi keretrendszereket alkalmaztuk:

- **Backend (ASP.NET Core Web API)** – *xUnit*: az üzleti logika egységtesztjeihez.
- **React webes kliens** – *Vitest*.
- **WPF asztali kliens** – *MSTest*: a felhasználói felület és a backend kommunikáció funkcionális tesztelésére.

A tesztelés során a rendszer minden komponense megfelelően működött, és stabil kommunikációt biztosított a backend szolgáltatással.

Részletesebb képernyőképek és további tesztesetek a Mellékletben (lásd: 7.3) találhatók.

6. fejezet

Rendszerkövetelmények és telepítés

6.1. Webes alkalmazás

A webalkalmazás használatához az alábbi feltételek szükségesek:

- Modern webböngésző (Chrome, Edge, Firefox)
- Internetkapcsolat
- JavaScript engedélyezése

Megjegyzés: A jelenlegi verzió fejlesztői környezetben, lokálisan fut. Éles szerverre történő telepítés nem történt, így a webalkalmazás kizárólag a helyi gépen érhető el a fejlesztői szerver futtatásával (pl. localhost).

6.1.1. WPF Desktop alkalmazás

Az adminisztrációs alkalmazás futtatásához az alábbi feltételek szükségesek:

- Windows operációs rendszer
- .NET futtatókörnyezet
- Internetkapcsolat az API eléréséhez

A telepítés futtatható állomány segítségével történik. A rendszer szintén lokális környezetben fut, az adatok eléréséhez a helyi API szerver futtatása szükséges.

6.1.2. Megjegyzés a lokális futtatásról

A rendszer lokális telepítésben történő futtatása azt jelenti, hogy:

- A webes felület kizárólag a helyi gépen, fejlesztői környezetből érhető el.

- Az API és az adatbázis szintén helyileg kerül telepítésre és futtatásra.
- Éles, publikus elérés nem került kialakításra.
- A lokális futtatás előnye, hogy a rendszer teljes funkcionalitása tesztelhető külső szerver igénybevétele nélkül.

6.1.3. A rendszer elindításának folyamata

A rendszer teljes körű működéséhez mind a backend (API), mind a frontend szolgáltatások elindítása szükséges.

6.1.4. Backend indítása

- A backend mappába történő navigálás terminál segítségével: `cd .\backend\PhoneScout_GitHub\`
- A backend szolgáltatás indítása: `dotnet run .\PhoneScout_GitHub.csproj`
- Sikeres indítást követően a Swagger felület a `http://localhost:5175/swagger` címen érhető el

6.1.5. Webes frontend indítása

- A frontend mappába történő navigálás terminál segítségével: `cd .\frontend\`
- Az első futtatás előtt a szükséges függőségek telepítése: `npm install`
- A projekt indítása: `npm start`
- A rendszer a böngészőben, localhost címen válik elérhetővé

6.1.6. Adminisztrációs felület (WPF) használata

A WPF alapú adminisztrációs alkalmazás indítására és telepítésére több lehetőség áll rendelkezésre a környezettől függően.

Indítás fejlesztői környezetből (Visual Studio)

- A projekt `.sln` fájljának megnyitása a Visual Studio alkalmazásban
- A WPF projekt beállítása *Startup Project*-ként
- Az alkalmazás indítása a *Start* gombbal vagy az `F5` billentyű segítségével

Telepítés telepítőcsomag segítségével

A telepítőcsomag (`.exe` vagy `.msi`) a `PhoneScoutAdminInstaller/Debug` mappában található.

- A telepítő futtatásával a varázsló végigvezeti a felhasználót a telepítési folyamaton
- A telepítést követően az alkalmazás regisztrálásra kerül a rendszerben
- Az alkalmazás az API szerver futása esetén azonnal használatba vehető

6.1.7. Adatbázis előkészítése lokális futtatáshoz

Az adatbázis mérete miatt lokális futtatás esetén az adatbáziskezelő beállításainak módosítása szükséges az alábbiak szerint.

Az Apache konfigurációjában található `php.ini` fájl módosításai:

- `upload_max_filesize` értéke: 256M
- `post_max_size` értéke: 256M
- `memory_limit` értéke: 512M
- `max_execution_time` értéke: 300
- `max_input_time` értéke: 300

A MySQL konfigurációjában található `my.ini` fájl módosítása:

- `max_allowed_packet` értéke: 256M

A konfiguráció módosítását követően a mellékletben található kód segítségével felhasználói fiók hozható létre a phpMyAdmin felületén.

7. fejezet

Összegzés

7.1. Projekt értékelése

A PhoneScout projekt célja egy modern, többplatformos telefonbolt és szervizrendszer megvalósítása volt. A kitűzött célok megvalósultak, a rendszer funkcionálisan teljes, és megfelel a korábban meghatározott követelményeknek.

A rétegzett architektúra, a REST alapú kommunikáció, valamint az objektum-relációs leképezés alkalmazása biztosítja a rendszer strukturált és karbantartható működését.

A frontend és backend szétválasztása lehetővé teszi a későbbi bővítést és skálázást, míg az adminisztrációs WPF alkalmazás hatékony adatkezelést biztosít.

7.2. Továbbfejlesztési lehetőségek

A rendszer jövőbeni fejlesztési irányai lehetnek:

- Az oldal több nyelven elérhetővé tétele.
- Sötét és világos megjelenés az oldalhoz
- Telefonépítő funkció. (Szűrés továbbfejlesztése)

Összességében a projekt sikeresen demonstrálja egy komplex, valós üzleti problémát megoldó webalkalmazás teljes életciklusát a tervezéstől a megvalósításon át a tesztelésig.

Köszönetnyilvánítás

Szeretnénk köszönetet mondani mindazoknak, akik segítették vizsgaremekünk létrejöttét. Külön köszönettel tartozunk konzulensünknek, Kerényi Róbert Nándornak, a szakmai iránymutatásért, a hasznos tanácsokért és a projekt elkészítése során nyújtott támogatásért.

Köszönettel tartozunk továbbá Négyesi Gábor, Deák Csaba, Kasza László Róbert és Lázár Zsolt oktatóinknak, akik tanácsaikkal és támogatásukkal hozzájárultak a munka sikeres megvalósításához.

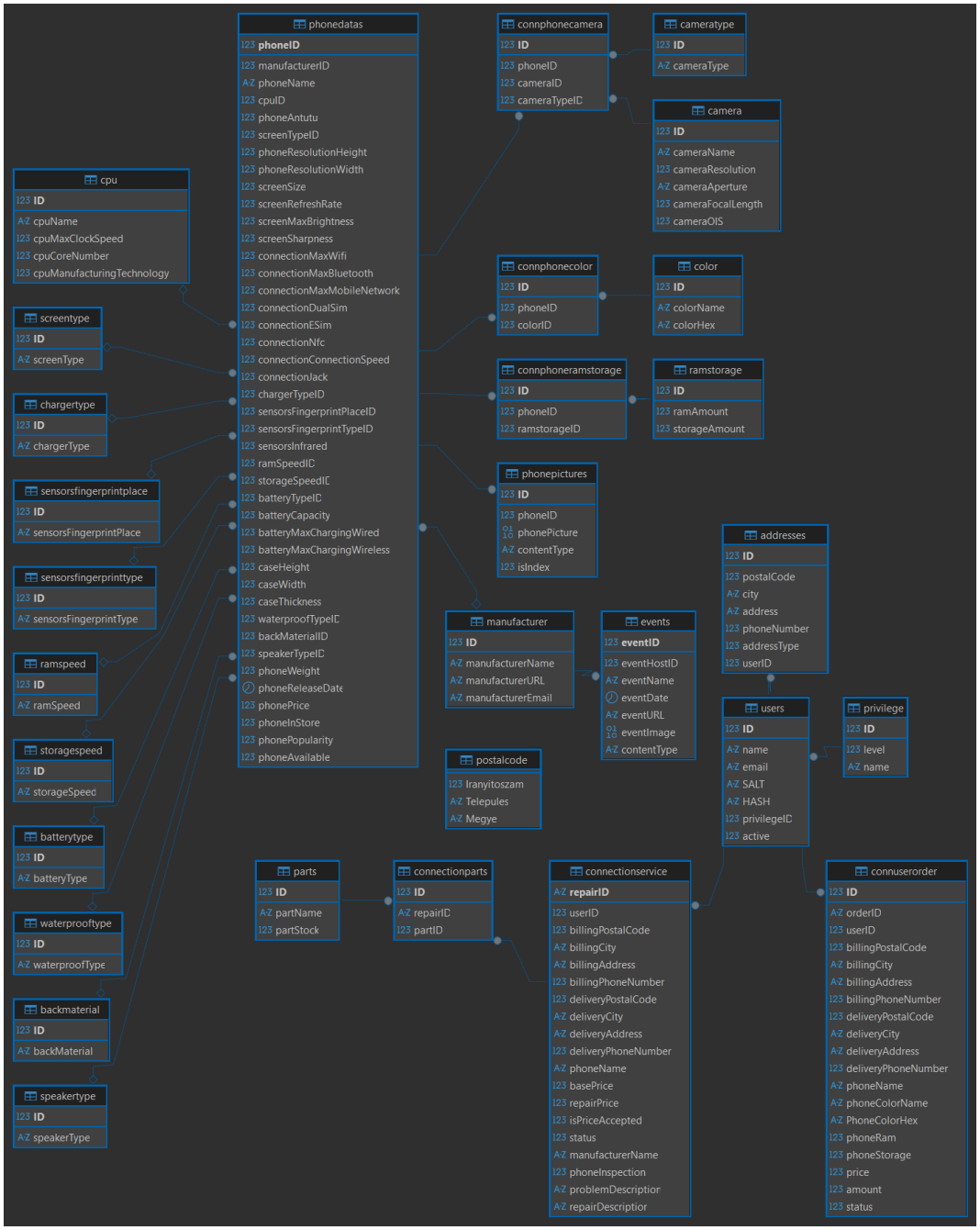
Emellett köszönjük mindazok segítségét, akik hozzájárultak a projekt elkészítéséhez.

Mellékletek

Xiaomi 13T részletes specifikációja

CPU / Processzor	
Név / Típus	Mediatek Dimensity 8200 Ultra
Magok	8 mag
Órajel	3 GHz
Technológia	4 nm
Antutu	905,365 pont

7.1. ábra. A telefon oldalon lévő grafika, működés közben.



7.2. ábra. Az adatbázis EK-diagramja.

```
vi.mock("axios");

const renderWithRouter = (component) => {
  return render(<BrowserRouter>{component}</BrowserRouter>);
};

describe("PhoneCard komponens tesztek", () => {
  beforeEach(() => {
    localStorage.clear();
    vi.clearAllMocks();
  });

  test("betölti a telefon képét az API-ból", async () => {
    const mockBlob = new Blob(["fake image"], { type: "image/jpeg" });
    axios.get.mockResolvedValueOnce({
      status: 200,
      data: mockBlob,
    });

    renderWithRouter(
      <PhoneCard
        phoneId={5}
        phoneName="iPhone 15"
        phoneInStore={true}
        phonePrice={399990}
        colors={[]}
        ramStoragePairs={[]}
      />
    );

    await waitFor(() => {
      expect(axios.get).toHaveBeenCalledWith(
        "http://localhost:5175/api/blob/GetIndex/5",
        { responseType: "blob" }
      );
    });
  });
});
```

7.3. ábra. Frontend telefonbetöltés teszt.

```
✓ src/components/PhoneCard.test.jsx (1 test) 88ms
✓ src/pages/Home.test.jsx (1 test) 123ms
stdout | src/components/Login.test.jsx > Login komponens tesztek > sikeres bejelentkezés végrehajt egy navigációt
sikeres bejelentkezés.

✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms
✓ src/pages/Home.test.jsx (1 test) 123ms
stdout | src/components/Login.test.jsx > Login komponens tesztek > sikeres bejelentkezés végrehajt egy navigációt
sikeres bejelentkezés.

✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms
stdout | src/components/Login.test.jsx > Login komponens tesztek > sikeres bejelentkezés végrehajt egy navigációt
sikeres bejelentkezés.

✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms
sikeres bejelentkezés.

✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms

✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms
✓ src/components/Login.test.jsx (1 test) 375ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms
    ✓ sikeres bejelentkezés végrehajt egy navigációt  368ms

Test Files  4 passed (4)
Test Files  4 passed (4)
     Tests  5 passed (5)
  Start at  11:19:05
  Duration  5.05s (transform 855ms, setup 1.27s, import 4.42s, tests 662ms, environment 10.30s)
```

7.4. ábra. Frontend teszt eredmények.

address

GET /api/address/GetAddresses/{userID}/{addressType} Egy profilhoz tartozó címek lekérése.

POST /api/address/PostAddress Egy profilhoz tartozó cím feltöltése.

Az adatok megadása után a felhasználóhoz rendelt cím feltöltésre kerül.

Parameters

No parameters

Request body

```
{
  "id": 0,
  "postalCode": 3525,
  "city": "Miskolc",
  "address": "Kandó Kálmán u. 10",
  "phoneNumber": 36204206769,
  "addressType": 0,
  "userID": 13
}
```

Request URL

http://localhost:5175/api/address/PostAddress

Server response

Code	Details
200	Response body: A cím sikeresen felvéve.

Response headers

```
access-control-allow-origin: *
content-type: text/plain; charset=utf-8
date: Wed,11 Mar 2026 08:18:25 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
200	OK

7.5. ábra. Swagger cím feltöltés teszt, jó paraméterekkel.

PUT /api/address/PutAddress/{addressId} Egy profilhoz tartozó cím módosítása.

Az elérési útban a cím azonosítóját adjuk meg, és a testben található adatokkal frissítjük.

Parameters

Name	Description
addressId * required integer($int32) (path)	14

Request body

```
{
  "id": 14,
  "postalCode": 1111,
  "city": "Budapest",
  "address": "Módosított u. 10",
  "phoneNumber": 36701234567,
  "addressType": 1,
  "userID": 13
}
```

Execute

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5175/api/address/PutAddress/14' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 14,
  "postalCode": 1111,
  "city": "Budapest",
  "address": "Módosított u. 10",
  "phoneNumber": 36701234567,
  "addressType": 1,
  "userID": 13
}'
```

Request URL

```
http://localhost:5175/api/address/PutAddress/14
```

Server response

Code	Details
200	Response body A cím sikeresen frissítve. Response headers access-control-allow-origin: * content-type: text/plain; charset=utf-8 date: Wed,11 Mar 2026 10:44:40 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description
200	OK

7.6. ábra. Swagger cím módosítás teszt, jó paraméterekkel.

PUT /api/address/PutAddress/{addressId} Egy profilhoz tartozó cím módosítása.

Az elérési útban a cím azonosítóját adjuk meg, és a testben található adatokkal frissítjük.

Parameters

Name	Description
addressId * required integer($int32) (path)	16

Request body

```
{
  "id": 14,
  "postalCode": 1111,
  "city": "Budapest",
  "address": "Módosított u. 10",
  "phoneNumber": 36701234567,
  "addressType": 1,
  "userID": 13
}
```

Execute

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5175/api/address/PutAddress/16' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 14,
  "postalCode": 1111,
  "city": "Budapest",
  "address": "Módosított u. 10",
  "phoneNumber": 36701234567,
  "addressType": 1,
  "userID": 13
}'
```

Request URL

```
http://localhost:5175/api/address/PutAddress/16
```

Server response

Code	Details
404 Undocumented	Error: Not Found Response body A cím nem található vagy nem jogosult a módosításra. Response headers access-control-allow-origin: * content-type: text/plain; charset=utf-8 date: Wed,11 Mar 2026 10:48:04 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description
200	OK

7.7. ábra. Swagger cím módosítás teszt, nem létező cím esetén.

DELETE /api/address/DeleteAddress/{addressId} Egy profilhoz tartozó cím törlése.

Az elérési útban a cím azonosítóját adjuk meg, majd a rekord törlésre kerül.

Parameters

Name	Description
addressId * required integer($int32) (path)	14

Execute

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5175/api/address/DeleteAddress/14' \
  -H 'accept: */*'
```

Request URL

http://localhost:5175/api/address/DeleteAddress/14

Request URL

http://localhost:5175/api/address/DeleteAddress/14

Server response

Code	Details
200	Response body A cím sikeresen törölve. Response headers access-control-allow-origin: * content-type: text/plain; charset=utf-8 date: Wed,11 Mar 2026 10:50:56 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description
200	OK

7.8. ábra. Swagger cím törlése teszt, jó paraméterekkel.

DELETE /api/address/DeleteAddress/{addressId} Egy profilhoz tartozó cím törlése.

Az elérési útban a cím azonosítóját adjuk meg, majd a rekord törlésre kerül.

Parameters

Name	Description
addressId * required integer($int32) (path)	14

Execute

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5175/api/address/DeleteAddress/14' \
  -H 'accept: */*'
```

Request URL

http://localhost:5175/api/address/DeleteAddress/14

Request URL

http://localhost:5175/api/address/DeleteAddress/14

Server response

Code	Details
404 Undocumented	Error: Not Found Response body A cím nem található. Response headers access-control-allow-origin: * content-type: text/plain; charset=utf-8 date: Wed,11 Mar 2026 10:53:00 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description
200	OK

7.9. ábra. Swagger cím törlése teszt, nem létező cím esetén.

address

GET /api/address/GetAddresses/{userID}/{addressType} Egy profilhoz tartozó címek lekérése.

Az elérési útban a felhasználó azonosítóját, és a cím típusát kell megadni, ami lehet: 0 - számlázási | 1 - szállítási.

Parameters

Name	Description
userID * required integer($int32) (path)	13
addressType * required integer($int32) (path)	0

Execute

Code Details

200 Response body

```
[
  {
    "id": 14,
    "postalCode": 3525,
    "city": "Miskolc",
    "address": "Kandó Kálmán u. 10",
    "phoneNumber": 36204206769,
    "addressType": 0,
    "userID": 13
  }
]
```

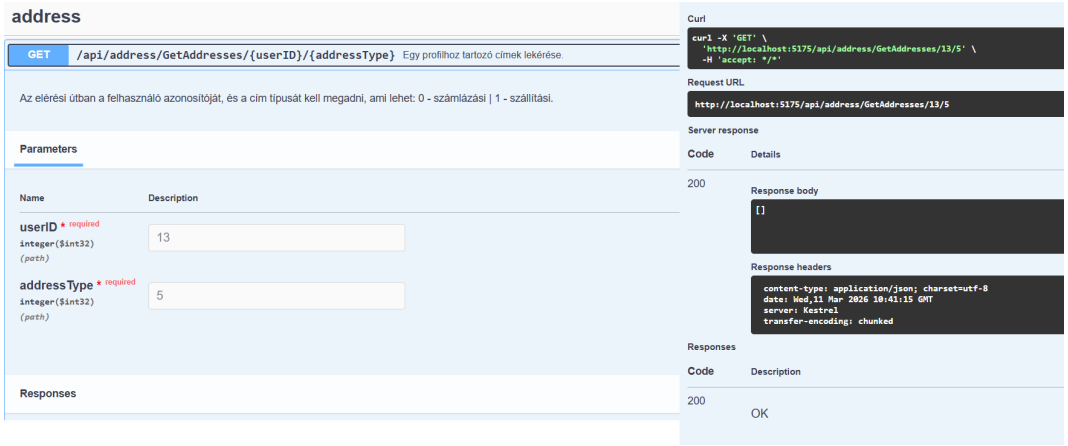
Response headers

```
content-type: application/json; charset=utf-8
date: Wed,11 Mar 2026 08:23:56 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
200	OK

7.10. ábra. Swagger egy cím lekérése teszt, jó paraméterekkel.



7.11. ábra. Swagger egy cím lekérése teszt, nem létező cím esetén.

```
[Fact]
0 references
public void autoFillPCC_InvalidPostalCode_ReturnsBadRequest()
{
    var data = new List<Postalcode>
    {
        new Postalcode { Iranyitoszam = 1051, Telepules = "Budapest" }
    };

    var controller = GetControllerWithData(data);

    var dto = new PostalCodeDTO
    {
        postalCode = 9999,
        city = ""
    };

    var result = controller.autoFillPCC(dto);

    var badRequest = Assert.IsType<BadRequestObjectResult>(result);

    Assert.Equal("Az irányítószám nem található.", badRequest.Value);
}
```

7.12. ábra. Backend város kiegészítő controller tesztelése, rossz irányítószám esetén. Eredmény sikeres.

```
[Fact]
0 references
public void autoFillPCC_ValidPostalCode_ReturnsOk()
{
    var data = new List<Postalcode>
    {
        new Postalcode { Iranyitoszam = 1051, Telepules = "Budapest" }
    };

    var controller = GetControllerWithData(data);

    var dto = new PostalCodeDTO
    {
        postalCode = 1051,
        city = ""
    };

    var result = controller.autoFillPCC(dto);

    var okResult = Assert.IsType<OkObjectResult>(result);
    var value = Assert.IsType<Postalcode>(okResult.Value);

    Assert.Equal("Budapest", value.Telepules);
}
```

7.13. ábra. Backend város kiegészítő controller tesztelése, jó paraméterek esetén. Eredmény sikeres.

```
[Fact]
0 references
public void PostRepair_UserNotFound_ReturnsBadRequest()
{
    var controller = GetController(new List<User>(), new List<Part>());

    var dto = new profileRepairPostDTO
    {
        userID = 1,
        parts = new List<string> { "Képernyő" }
    };

    var result = controller.postRepair(dto);

    var badRequest = Assert.IsType<BadRequestObjectResult>(result);

    Assert.Equal("A felhasználó nem található.", badRequest.Value);
}
```

7.14. ábra. Backend szervizelés feltöltés controller tesztelése, nem létező felhasználó esetén. Eredmény sikeres.

```
[Fact]
0 references
public void PostRepair_NoParts_ReturnsBadRequest()
{
    var users = new List<User>
    {
        new User { Id = 1 }
    };

    var controller = GetController(users, new List<Part>());

    var dto = new profileRepairPostDTO
    {
        userID = 1,
        parts = new List<string>()
    };

    var result = controller.postRepair(dto);

    var badRequest = Assert.IsType<BadRequestObjectResult>(result);

    Assert.Equal("Nincsenek megadva alkatrészek.", badRequest.Value);
}
```

7.15. ábra. Backend szervizelés feltöltés controller tesztelése, alkatrészlista nélkül. Eredmény sikeres.

```
[Fact]
0 references
public void PostRepair_ValidRequest_ReturnsOk()
{
    var users = new List<User>
    {
        new User { Id = 1 }
    };

    var parts = new List<Part>
    {
        new Part { Id = 1, PartName = "Kamera" }
    };

    var controller = GetController(users, parts);

    var dto = new profileRepairPostDTO ···

    var result = controller.postRepair(dto);

    var ok = Assert.IsType<OkObjectResult>(result);

    Assert.Equal("A szerviz igénylés befejeződött.", ok.Value);
}
```

7.16. ábra. Backend szervizelés feltöltés controller tesztelése, jó paraméterek esetén. Eredmény sikeres.

```
PhoneScout_GitHub_Tests test succeeded (2.2s)

Test summary: total: 11; failed: 0; succeeded: 11; skipped: 0; duration: 2.2s
```

7.17. ábra. Backend teszteredmények.

```
[Fact]
0 references
public void UpdateRepair_RepairNotFound_ReturnsNotFound()
{
    var controller = GetController(new List<User>(),
    new List<Part>(), new List<Connectionservice>(),
    new List<Connectionpart>());

    var dto = new profileRepairPostDTO { userID = 1, parts = new List<string> { "Képernyő" } };
    var result = controller.updateRepair(dto, "nonexistent");

    var notFound = Assert.IsType<NotFoundObjectResult>(result);
    Assert.Equal("A szerviz igénylés nem található.", notFound.Value);
}
```

7.18. ábra. Backend szervizelés módosítás controller tesztelése, nem létező szervizelés esetén. Eredmény sikeres.

```
[Fact]
0 references
public void UpdateRepair_UserNotFound_ReturnsBadRequest()
{
    var repairs = new List<Connectionservice> { new Connectionservice { RepairId = "R1" } };
    var controller = GetController(new List<User>(), new List<Part>(), repairs, new List<Connectionpart>());

    var dto = new profileRepairPostDTO { userID = 1, parts = new List<string> { "Screen" } };
    var result = controller.updateRepair(dto, "R1");

    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    Assert.Equal("A felhasználó nem található.", badRequest.Value);
}
```

7.19. ábra. Backend szervizelés módosítás controller tesztelése, nem létező felhasználó esetén. Eredmény sikeres.

```
[Fact]
0 references
public void UpdateRepair_NoParts_ReturnsBadRequest()
{
    var users = new List<User> { new User { Id = 1 } };
    var repairs = new List<Connectionservice> { new Connectionservice { RepairId = "R1" } };
    var controller = GetController(users, new List<Part>(), repairs, new List<Connectionpart>());

    var dto = new profileRepairPostDTO { userID = 1, parts = new List<string>() };
    var result = controller.updateRepair(dto, "R1");

    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    Assert.Equal("Nincsenek megadva alkatrészek.", badRequest.Value);
}
```

7.20. ábra. Backend szervizelés módosítás controller tesztelése, alkatrészlista nélkül. Eredmény sikeres.

```
[Fact]
0 references
public void UpdateRepair_ValidUpdate_ReturnsOk()
{
    var users = new List<User> { new User { Id = 1 } };
    var parts = new List<Part> { new Part { Id = 1, PartName = "Screen" } };
    var repairs = new List<Connectionservice> { new Connectionservice { RepairId = "R1", UserId = 1 } };
    var repairParts = new List<Connectionpart>();

    var controller = GetController(users, parts, repairs, repairParts);

    var dto = new profileRepairPostDTO
    {
        userID = 1,
        parts = new List<string> { "Screen" },
        billingPostalCode = 1051,
        billingCity = "Budapest"
    };

    var result = controller.updateRepair(dto, "R1");

    var ok = Assert.IsType<OkObjectResult>(result);
    Assert.Equal("A szerviz igénylés befejeződött.", ok.Value);
}
```

7.21. ábra. Backend szervizelés módosítás controller tesztelése, jó paraméterek esetén. Eredmény sikeres.

```
[TestMethod]
0 references
public void PriceIsNotChanged()
{
    decimal originalPrice = 100m;
    decimal currentPrice = 100m;

    bool isChanged = currentPrice != originalPrice;

    Assert.IsFalse(isChanged);
}

[TestMethod]
0 references
public void PriceIsChanged()
{
    decimal originalPrice = 100m;
    decimal currentPrice = 150m;

    bool isChanged = currentPrice != originalPrice;

    Assert.IsTrue(isChanged);
}
```

7.22. ábra. WPF teszt: Szerviz módosítás ár változás ellenőrzés.

PhoneScoutAdminTests (5)	7 ms
<Empty Namespace> (2)	6 ms
PriceTests (2)	6 ms
PriceIsChanged	6 ms
PriceIsNotChanged	< 1 ms
PhoneScoutAdmin.ViewModels.Tests	1 ms
PartsTests (3)	1 ms
PartsAreChanged	1 ms
PartsAreChanged_WhenCountIs...	< 1 ms
PartsAreNotChanged	< 1 ms

7.23. ábra. WPF teszteredmények.